

CASE STUDY: APPLICATION OF RNNs IN SENTIMENT ANALYSIS FOR A SMARTWATCH COMPANY

Introduction

With the proliferation of online platforms, user reviews have become a gold mine for businesses to understand customer sentiment. For our fictional smartwatch company, these reviews can offer insights into product performance, features, and areas of improvement. The sentiment analysis of such reviews can help the company in formulating strategies, improving product quality, and targeting marketing efforts more effectively.

Problem Description

The fictional smartwatch company has collected a substantial number of reviews from Google. The challenge is to analyze these reviews to classify sentiments as positive, neutral, or negative. Manual evaluation is time-consuming and not scalable. Therefore, an automated solution is needed.

Methodology and Approach

To automate the sentiment analysis, we will utilize Recurrent Neural Networks (RNNs). RNNs are adept at handling sequences (like sentences in reviews) and can maintain information from previous inputs, making them suitable for this task.

Sample review data: The dataset consists of two columns: Review, which contains textual feedback about a smartwatch, and Sentiment, labeled as 0 (negative), 1 (neutral), or 2 (positive).

```
import pandas as pd
import numpy as np
df = pd.read_csv('./smartwatch_reviews.csv')
df.head(5)
```

Output:

	Review	Sentiment
0	No native music streaming capability.	0
1	Won't make you go 'wow', but won't disappoint ...	1
2	Doesn't track my steps accurately.	0
3	It's more than just a watch, it's a lifestyle ...	2
4	The touch screen is very responsive and fluid.	2

RNN implementation code with simple explanation

Step 1 : Importing Libraries : Using TensorFlow, a toolset for building machine learning models, and fetching tools for text processing.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Step 2 : For tokenization and padding, we utilize a tokenizer that translates words into numbers based on the 10,000 most common words, labeling any unseen words as ``<OOV>``. By applying ``fit_on_texts``, we familiarize the tokenizer with our reviews. Once familiar, it transforms these reviews into lists of numbers using ``texts_to_sequences``. To ensure consistency in review length, we employ ``pad_sequences`` which standardizes them by adding zeros as needed.

```
# Tokenization and Padding
tokenizer = Tokenizer(num_words=10000, oov_token="<OOV>")
tokenizer.fit_on_texts(df['Review'])
sequences = tokenizer.texts_to_sequences(df['Review'])
padded = pad_sequences(sequences, padding='post')
```

Step 3: Data Split - we allocate 80% for training, analogous to studying, and the remaining 20% for testing, much like an unseen exam.

```
# Splitting data into training and testing
split = int(0.8 * len(padded))
train_data = padded[:split]
test_data = padded[split:]
train_labels = df['Sentiment'][:split]
test_labels = df['Sentiment'][split:]
```

Step 4 : Build the RNN model - we start with the Embedding Layer, which morphs numbers representing words into detailed feature lists. Following

that, the SimpleRNN layers act as the model's memory, retaining critical portions of the reviews to grasp the context. Finally, the Dense Layer assesses the sentiment of the review after its thorough examination.

```
# RNN model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 16, input_length=padded.shape[1]),
    tf.keras.layers.SimpleRNN(32, return_sequences=True),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(3, activation='softmax')
])
```

Step 5: Compiling the Model - Prepares model for training. Specifies how to measure mistakes (`loss`), improve (`optimizer`), and tracks its accuracy (`metrics`).

```
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Step 6 : Training - Teaches the model using the training set over 10 rounds, testing its knowledge after each round.

```
history_rnn = model.fit(train_data, train_labels, epochs=10,
validation_data=(test_data, test_labels))
```

```
Epoch 1/10
13/13 [=====] - 2s 32ms/step - loss: 1.0325 - accuracy: 0.4650 - val_loss: 0.9064 - val_accuracy: 0.6900
Epoch 2/10
13/13 [=====] - 0s 10ms/step - loss: 0.6708 - accuracy: 0.8375 - val_loss: 0.5283 - val_accuracy: 0.8500
Epoch 3/10
13/13 [=====] - 0s 9ms/step - loss: 0.3208 - accuracy: 0.9275 - val_loss: 0.2539 - val_accuracy: 0.9600
Epoch 4/10
13/13 [=====] - 0s 9ms/step - loss: 0.1180 - accuracy: 0.9900 - val_loss: 0.1263 - val_accuracy: 0.9700
Epoch 5/10
13/13 [=====] - 0s 9ms/step - loss: 0.0457 - accuracy: 0.9975 - val_loss: 0.0962 - val_accuracy: 0.9700
Epoch 6/10
13/13 [=====] - 0s 9ms/step - loss: 0.0207 - accuracy: 1.0000 - val_loss: 0.0964 - val_accuracy: 0.9700
Epoch 7/10
13/13 [=====] - 0s 10ms/step - loss: 0.0126 - accuracy: 1.0000 - val_loss: 0.0864 - val_accuracy: 0.9700
Epoch 8/10
13/13 [=====] - 0s 17ms/step - loss: 0.0088 - accuracy: 1.0000 - val_loss: 0.0843 - val_accuracy: 0.9700
Epoch 9/10
13/13 [=====] - 0s 19ms/step - loss: 0.0068 - accuracy: 1.0000 - val_loss: 0.0832 - val_accuracy: 0.9700
Epoch 10/10
13/13 [=====] - 0s 18ms/step - loss: 0.0056 - accuracy: 1.0000 - val_loss: 0.0857 - val_accuracy: 0.9700
```

Challenges in Implementation and Strategies to Tackle Them:

Vocabulary Size: The real-world reviews might have a vast vocabulary.

```
tokenizer = Tokenizer(num_words=10000, oov_token="<OOV>")
```

Here, we limit the vocabulary to 10,000 words and represent out-of-vocabulary words with ``<OOV>``. The fixed size of the tokenizer's vocabulary can miss some words, leading to limited understanding. Increase the vocabulary size or employ techniques like subword tokenization or embeddings like Word2Vec or GloVe.

Variable Review Length: Reviews can be of varying lengths.

```
padded = pad_sequences(sequences, padding='post')
```

Padding ensures reviews have the same length, adding zeros where necessary.

Vanishing Gradient Problem: RNNs are often affected by the vanishing gradient problem, making it difficult for the model to learn long-range dependencies. Use architectures like LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Units) which are designed to tackle this issue.

```
# LSTM model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 16, input_length=padded.shape[1]),
    tf.keras.layers.LSTM(32, return_sequences=True), # First LSTM layer
    with return_sequences=True
    tf.keras.layers.LSTM(32), # Second LSTM layer
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
history_lstm = model.fit(train_data, train_labels, epochs=10,
validation_data=(test_data, test_labels))
```

```
Epoch 1/10
13/13 [=====] - 7s 73ms/step - loss: 1.0976 - accuracy: 0.3325 - val_loss: 1.0963 - val_accuracy: 0.3000
Epoch 2/10
13/13 [=====] - 0s 15ms/step - loss: 1.0863 - accuracy: 0.3575 - val_loss: 1.0809 - val_accuracy: 0.3000
Epoch 3/10
13/13 [=====] - 0s 13ms/step - loss: 1.0442 - accuracy: 0.4225 - val_loss: 0.9976 - val_accuracy: 0.4800
Epoch 4/10
13/13 [=====] - 0s 14ms/step - loss: 0.8457 - accuracy: 0.6375 - val_loss: 0.6806 - val_accuracy: 0.8100
Epoch 5/10
13/13 [=====] - 0s 13ms/step - loss: 0.5108 - accuracy: 0.8650 - val_loss: 0.4510 - val_accuracy: 0.8900
Epoch 6/10
13/13 [=====] - 0s 13ms/step - loss: 0.2228 - accuracy: 0.9675 - val_loss: 0.3007 - val_accuracy: 0.8900
Epoch 7/10
13/13 [=====] - 0s 13ms/step - loss: 0.0729 - accuracy: 0.9825 - val_loss: 0.2226 - val_accuracy: 0.9500
Epoch 8/10
13/13 [=====] - 0s 22ms/step - loss: 0.0249 - accuracy: 0.9975 - val_loss: 0.2323 - val_accuracy: 0.9500
Epoch 9/10
13/13 [=====] - 0s 14ms/step - loss: 0.0121 - accuracy: 1.0000 - val_loss: 0.2512 - val_accuracy: 0.9500
Epoch 10/10
13/13 [=====] - 0s 13ms/step - loss: 0.0078 - accuracy: 1.0000 - val_loss: 0.2631 - val_accuracy: 0.9500
```

Overfitting: RNNs might overfit on training data if the dataset is not sufficiently large. Introduce dropout layers or use regularization techniques.

```
from keras.layers import Dropout
# LSTM model with Dropout
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 16, input_length=padded.shape[1]),
    tf.keras.layers.LSTM(32, return_sequences=True), # First LSTM layer
    with return_sequences=True
        Dropout(0.5), # Dropout after first LSTM
    tf.keras.layers.LSTM(32), # Second LSTM layer
    Dropout(0.5), # Dropout after second LSTM
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
history_lstm_dropout = model.fit(train_data, train_labels, epochs=10,
validation_data=(test_data, test_labels))
```

```
Epoch 1/10
13/13 [=====] - 4s 71ms/step - loss: 1.0989 - accuracy: 0.3500 - val_loss: 1.0979 - val_accuracy: 0.3000
Epoch 2/10
13/13 [=====] - 0s 18ms/step - loss: 1.0917 - accuracy: 0.3975 - val_loss: 1.0900 - val_accuracy: 0.3000
Epoch 3/10
13/13 [=====] - 0s 17ms/step - loss: 1.0699 - accuracy: 0.4550 - val_loss: 1.0489 - val_accuracy: 0.5100
Epoch 4/10
13/13 [=====] - 0s 18ms/step - loss: 0.9518 - accuracy: 0.6500 - val_loss: 0.7978 - val_accuracy: 0.7100
Epoch 5/10
13/13 [=====] - 0s 15ms/step - loss: 0.6067 - accuracy: 0.7750 - val_loss: 0.4751 - val_accuracy: 0.8000
Epoch 6/10
13/13 [=====] - 0s 18ms/step - loss: 0.2942 - accuracy: 0.9300 - val_loss: 0.2570 - val_accuracy: 0.9100
Epoch 7/10
13/13 [=====] - 0s 16ms/step - loss: 0.1350 - accuracy: 0.9725 - val_loss: 0.2060 - val_accuracy: 0.9300
Epoch 8/10
13/13 [=====] - 0s 15ms/step - loss: 0.0647 - accuracy: 0.9950 - val_loss: 0.2222 - val_accuracy: 0.9300
Epoch 9/10
13/13 [=====] - 0s 13ms/step - loss: 0.0376 - accuracy: 0.9950 - val_loss: 0.2240 - val_accuracy: 0.9300
Epoch 10/10
13/13 [=====] - 0s 13ms/step - loss: 0.0243 - accuracy: 0.9975 - val_loss: 0.3501 - val_accuracy: 0.9200
```

Comparing Outputs:

To compare the outputs of the RNN, LSTM, and LSTM with dropout models, you can consider multiple aspects, including:

Model Accuracy: You can observe the final training and validation accuracy of each model. Higher accuracy usually means better performance but ensure that both training and validation accuracies are high to prevent overfitting.

Loss Curves: Plot the training and validation loss for each epoch and compare them. This will give you an idea of whether a model is overfitting (if validation loss starts to increase while training loss continues to decrease).

Overfitting: Check the difference between training and validation accuracy or loss. A large gap might suggest that the model is overfitting. In that case, regularized models (like LSTM with dropout) may show less overfitting than others.

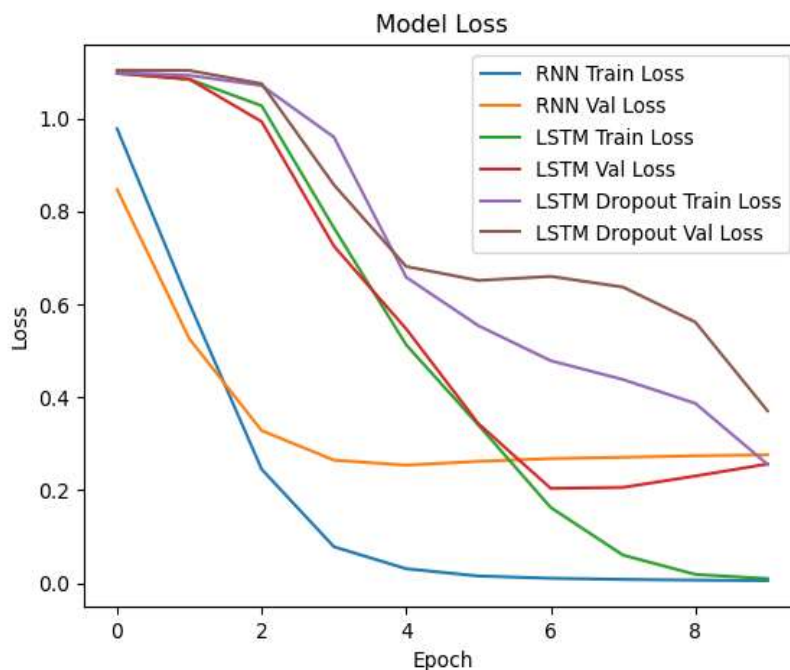
Convergence Speed: Notice how fast each model converges to a good result. Some models might give good performance but might require more epochs.

Model's Predictions: On a given test dataset, you can also compare the actual predictions. This might be insightful if you have certain examples that you think are particularly challenging or important.

Confusion Matrix: You can construct a confusion matrix for each model on the test data. This matrix will help you understand where the model's predictions are concentrated and if there are consistent misclassifications.

Loss Curves Implementation:

The plot displays the loss curves for RNN, LSTM, and LSTM with Dropout models:



RNN: Exhibits steady learning with a minor gap between training and validation loss, suggesting slight overfitting.

LSTM: While it learns rapidly, it shows signs of overfitting as validation loss increases after the 6th epoch.

LSTM with Dropout: Demonstrates balanced learning with stable validation loss, indicating good generalization and the effectiveness of dropout in reducing overfitting.

Choice for Deployment: Based on the plot, LSTM with Dropout is optimal for its generalization. However, the RNN is also a desirable choice, especially if resources or complexity are concerns.



Future Steps: Consider experimenting with varied dropout rates, other regularization methods, and tweaking hyperparameters like learning rate and batch size for better performance.

Remember, while RNNs can be a desirable choice for sentiment analysis, depending on the dataset size and complexity, other architectures or combinations of models might be more appropriate.

Conclusion:

RNNs offer a powerful method for sentiment analysis on user reviews. This approach allows our fictional smartwatch company to harness the vast data from online platforms and convert it into actionable insights. However, it's crucial to address challenges like vocabulary size and sequence length to ensure optimal performance. With continuous tuning and adaptation, this methodology can significantly impact product development and marketing strategies.